



DEPARTMENT OF ENGINEERING CYBERNETICS

TTK4550

Specialization Project

Authors:

Anders Haarberg Eriksen, Øyvind Stavdahl & Devon Rohlf

Date: 14th May 2026

Contents

1	Introduction	1
2	Theory	2
2.1	Anatomy	2
2.1.1	Anatomical directional terms	2
2.1.2	Anatomical terms of movement	2
2.1.3	Hand anatomy	3
2.2	Modelling	6
2.2.1	Homogenous transformations	6
2.3	Control	7
2.3.1	State-Space representation	7
2.3.2	Controllability	7
2.3.3	MRAC	7
3	Modelling	10
3.1	Finger model	10
3.1.1	Kinetic energy	12
3.1.2	Potential energy	13
3.1.3	Generalized forces	14
3.1.4	Rigid-body dynamics	14
3.2	Brushed DC Motor model	16
3.2.1	Estimating motor parameters	17
3.2.2	Estimating the armature inductance and rotor inertia	18
3.2.3	Mass-spring-damper load	21
3.2.4	State-space model of brushed DC motor with load	21
4	Controller design	22
4.1	Current controller	22
4.1.1	Discretization of current controller	23
4.1.2	Stability analysis and tuning of controller	23
4.1.3	Simulation of current controller	24

4.1.4	Evaluation of controller	27
4.2	Torque control	27
4.2.1	Force control with integrated fingertip force sensors	29
4.3	Model reference adaptive control (MRAC) for position control	29
4.3.1	Controllability of the system	30
4.3.2	MRAC implementation	30
4.3.3	Simulation of MRAC controller with finger dynamics	32
4.3.4	Cable tension	32
4.3.5	Updated dynamics with the cable tension	33
4.3.6	Evaluating model reference adaptive controller	34

List of Figures

2.1	The anatomical position, with the corresponding directional terms. Image from TeachMeAnatomy, 2026a	3
2.2	Anatomical terms of movement. Image from TeachMeAnatomy, 2026b	3
2.3	The bones of the hand. Image from author.	4
2.4	The joints of the hand. Image from author.	5
3.1	Schematic of the finger model, with three revolute joints, torsional springs and dampers.	11
3.2	Table of model parameters.	11
3.3	Schematic of the homogenous transformation frames.	13
3.4	Circuit diagram of a brushed DC motor.	16
3.5	Parameter identification of the armature inductance L_a and rotor inertia J_m using a step input in the armature voltage E_a	20
4.1	Armature current step response for discrete-time P+FF and P controllers tracking a 1 A reference, with $T_s = 0.2$ ms and initial motor speed $\omega_0 = 5$ rad s ⁻¹	25
4.2	Armature current step response for discrete-time P+FF and P controllers tracking a 1 A reference, with $T_s = 0.2$ ms and initial motor speed $\omega_0 = 0$ rad s ⁻¹	25
4.3	Effect of current measurement noise on the armature current response using a discrete-time P+FF controller with $T_s = 0.2$ ms.	26
4.4	Effect of current measurement noise on the armature current response using a discrete-time P+FF controller with $T_s = 1.0$ ms.	26
4.5	Torque control with a P controller when gripping a rigid object.	29
4.6	Force application on the finger. The flexion motor applies a force F_f on the flexion cable, which gets distributed to the three phalanges through the whiffle tree. Similarly, the extension motor applies a force F_e on the extension cable, which also gets distributed to the three phalanges through its own whiffle tree.	34

List of Tables

3.1	Constants in system matrices.	16
3.2	Relevant motor parameters extracted from the datasheet of the motor.	18

Chapter 1

Introduction

Hand exoskeletons have gained increasing attention in recent years due to their potential in rehabilitation, assistive technology, and human motion augmentation. By providing controlled support to the hand and fingers, such systems may help restore function in individuals with impaired motor ability, while also serving as platforms for studying human biomechanics and control. Because the hand is mechanically complex and highly dexterous, the development of effective hand exoskeletons requires accurate models of finger motion and dynamics.

A key challenge in the design of such systems is the interaction between the mechanical structure of the exoskeleton and the physiological properties of the human finger. The finger is a multi-joint system with coupled motion, passive elastic properties, and nonlinear dynamics. To design controllers that achieve stable and precise movement assistance, it is therefore important to establish a mathematical model that captures the essential kinematic and dynamic behavior of the finger, while remaining suitable for simulation and control design.

Chapter 2

Theory

2.1 Anatomy

The hand is a complex biomechanical system, comprised of many different bones, muscles, tendons, and ligaments. To explain different aspects of the hand in relation to one another, we use *anatomical terminology* [National Cancer Institute, 2026], or more specifically *anatomical directional terms*. Additionally when describing actions of muscles upon the skeleton, we use *anatomical terms of movement* [TeachMeAnatomy, 2026b].

2.1.1 Anatomical directional terms

Anatomical directional terms are used in relation to the *anatomical position*, shown in Figure 2.1, where the body is assumed standing upright, with arms at the sides and palms facing forward. Relevant terms include: *proximal*, *distal*.

- **Proximal** means toward or nearest the point of origin of a part.
- **Distal** means away from or farthest from the point of origin of a part.

2.1.2 Anatomical terms of movement

To explain the actions of muscles upon the skeleton, we use anatomical terms of movement, which describes the movement of a body part in relation to the anatomical position. Relevant terms include: *flexion*, *extension*, *abduction*, *adduction*.

- **Flexion** is a movement that decreases the angle between two body parts.
- **Extension** is a movement that increases the angle between two body parts.
- **Abduction** is a movement away from the midline of the body.

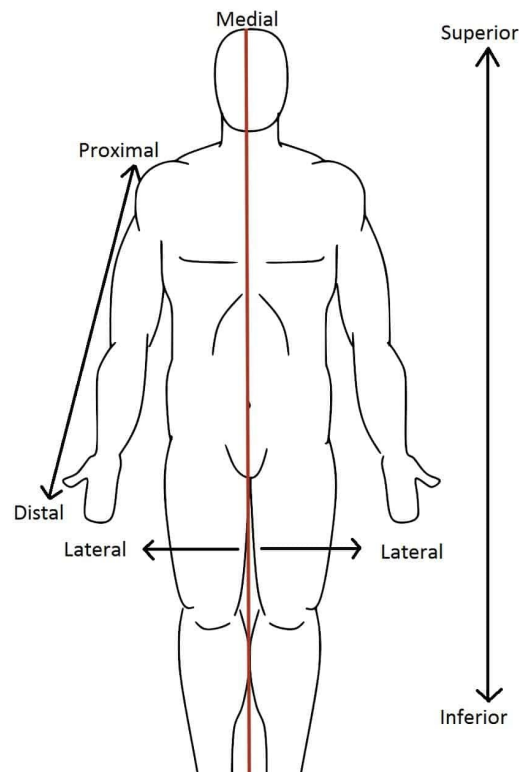


Figure 2.1: The anatomical position, with the corresponding directional terms. Image from TeachMeAnatomy, [2026a](#).

- **Adduction** is a movement toward the midline of the body.

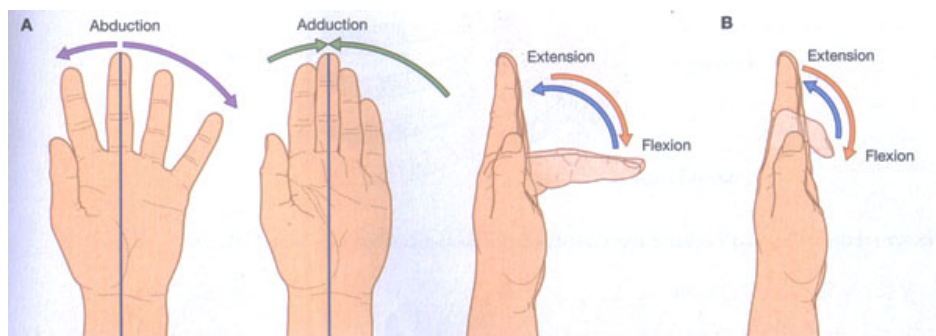


Figure 2.2: Anatomical terms of movement. Image from TeachMeAnatomy, [2026b](#).

2.1.3 Hand anatomy

The hand is comprised of several bones, muscles, tendons, and ligaments. The bones, illustrated in Figure 2.3, include the *carpals*, *metacarpals*, and *phalanges*. The carpals make out the wrist, the metacarpals make out the palm, and the phalanges make out the fingers. The thumb has two phalanges, while the other fingers have three. The bones of interest are the phalanges (marked in green), as they are the ones being moved by the exoskeleton. The metacarpals are assumed stationary for the task at hand, as the exoskeleton will provide stiffness to the palm and wrist.

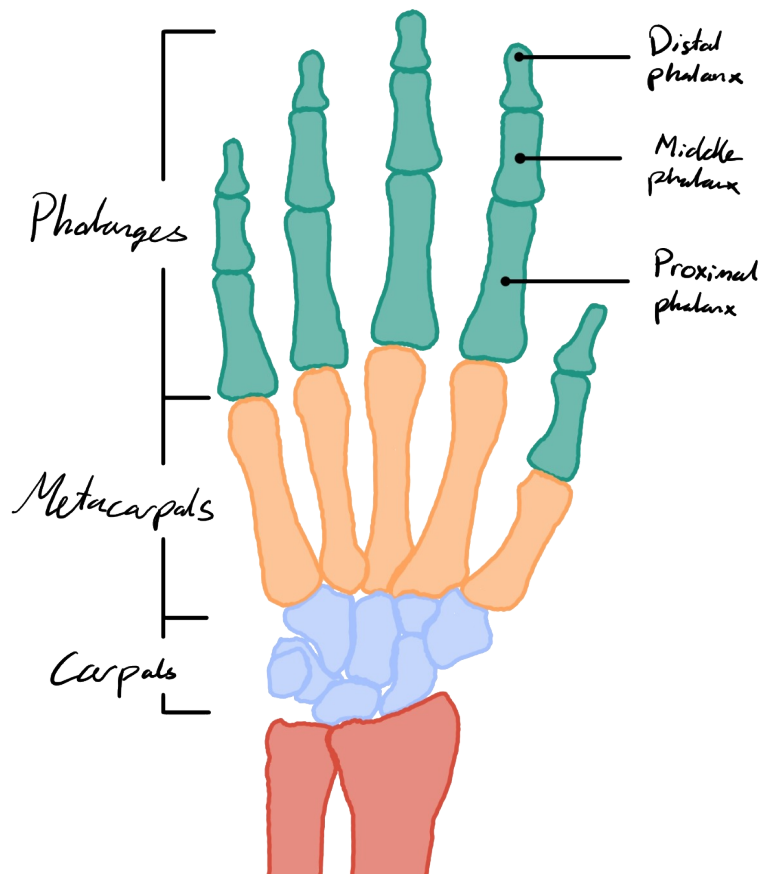


Figure 2.3: The bones of the hand. Image from author.

The phalanges are further divided into three sections: the *proximal phalanges*, the *middle phalanges*, and the *distal phalanges*. The proximal phalanges are the ones closest to the palm, while the distal phalanges are the ones furthest from the palm. The middle phalanges are the ones in between. The thumb does not have a middle phalanx, as it only has two phalanges.

Further details about the joints are shown in Figure 2.4. The joints of interest are the *metacarpophalangeal joints* (MCP), the *proximal interphalangeal joints* (PIP), and the *distal interphalangeal joints* (DIP). The MCP joints are the ones between the metacarpals and the proximal phalanges, while the PIP joints are the ones between the proximal phalanges and the middle phalanges, and the DIP joints are the ones between the middle phalanges and the distal phalanges. The thumb only has an MCP joint and an interphalangeal joint (IP), as it does not have a middle phalanx.

At each of the joints, there are collateral ligaments that provide stability. For the PIP, IP and DIP joints, they provide stiffness that prevents side to side motion. For the MCP joints they're relaxed when the finger is extended, facilitating for adduction and abduction, but stiff when the finger is flexed.

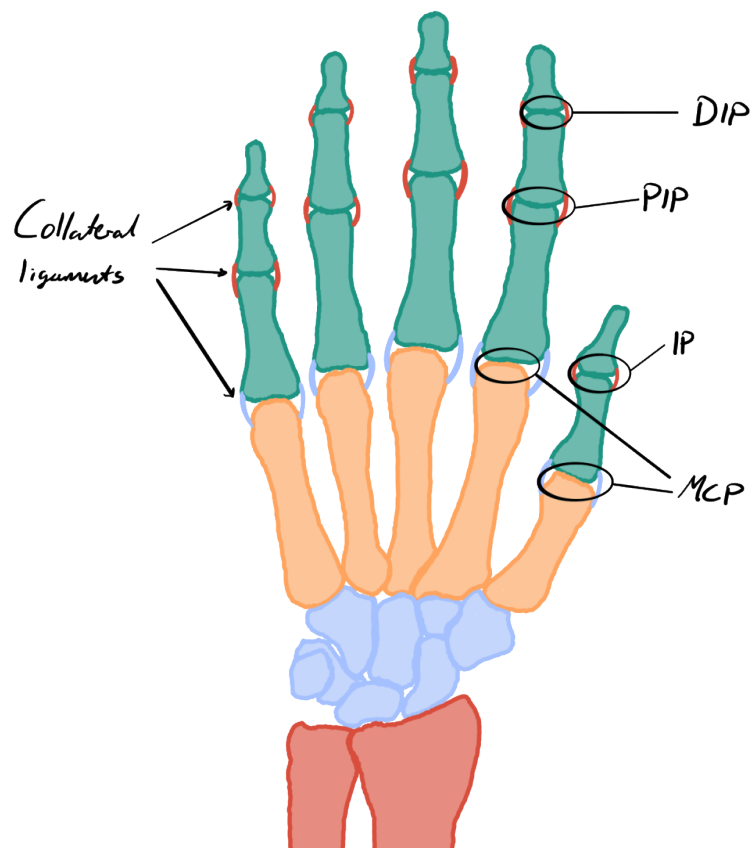


Figure 2.4: The joints of the hand. Image from author.

2.2 Modelling

2.2.1 Homogenous transformations

When modelling kinematic chains, we often need to move between frames. This often includes both rotation and translation. An efficient way to represent this is by using *homogenous transformations*.

Definition

For a 2D system, the rotation matrix for a counter-clockwise rotation by an angle θ is given by:

$$\mathbf{R}(\theta) = \begin{bmatrix} \cos(\theta) & -\sin(\theta) \\ \sin(\theta) & \cos(\theta) \end{bmatrix}. \quad (2.1)$$

Given a translation vector from frame i to frame j , written in terms of frame i , $\mathbf{d}_i^j = \begin{bmatrix} d_x^i \\ d_y^i \end{bmatrix}$, the homogenous transformation matrix from frame j to frame i , describing the rotation θ and translation $\mathbf{d}_i^j$ from frame i to frame j , can be expressed as:

$$\mathbf{H}_j^i = \begin{bmatrix} \mathbf{R}(\theta) & \mathbf{d}_i^j \\ \mathbf{0}_{1 \times 2} & 1 \end{bmatrix}. \quad (2.2)$$

Points represented in terms of homogenous transformations are represented as a 3D vector, where the first two elements represent the coordinates of the point in the plane, and the third element is always 1. For example, a point $\mathbf{p}_n$ in frame n with coordinates (x, y) can be represented as:

$$\mathbf{p}_n = \begin{bmatrix} p_x^n \\ p_y^n \\ 1 \end{bmatrix}. \quad (2.3)$$

Useful properties

Given a kinematic chain of size n , we can express the homogenous transformation from frame n to frame 1 as the product of the homogenous transformations between each consecutive frame:

$$\mathbf{H}_n^1 = \mathbf{H}_2^1 \mathbf{H}_3^2 \cdots \mathbf{H}_n^{n-1}. \quad (2.4)$$

Given a point $\mathbf{p}_n$ in frame n , we can express it in frame 1 as:

$$\mathbf{p}_1 = \begin{bmatrix} p_x^1 \\ p_y^1 \\ 1 \end{bmatrix} = \mathbf{H}_2^1 \mathbf{H}_3^2 \cdots \mathbf{H}_n^{n-1} \mathbf{p}_n = \mathbf{H}_n^1 \mathbf{p}_n. \quad (2.5)$$

2.3 Control

2.3.1 State-Space representation

A way to express a n 'th order ordinary differential equation (ODE) is to express it as a system of n first order ODEs. This is called the *state-space representation* of the system. A state-space representation commonly used in control theory for linear time-invariant (LTI) systems is the following:

$$\begin{aligned} \dot{\mathbf{x}}(t) &= \mathbf{A}\mathbf{x}(t) + \mathbf{B}\mathbf{u}(t) + \mathbf{d}(t), & \mathbf{x}(t) &\in \mathbb{R}^n, & \mathbf{u}(t) &\in \mathbb{R}^m, & \mathbf{y}(t) &\in \mathbb{R}^p, & \mathbf{d}(t) &\in \mathbb{R}^n, \\ \mathbf{y}(t) &= \mathbf{C}\mathbf{x}(t), & \mathbf{A} &\in \mathbb{R}^{n \times n}, & \mathbf{B} &\in \mathbb{R}^{n \times m}, & \mathbf{C} &\in \mathbb{R}^{p \times n}. \end{aligned} \quad (2.6)$$

Here, $\mathbf{x}(t)$ is the state vector, which contains all the information about the system at time t . $\mathbf{u}(t)$ is the input vector at time t . $\mathbf{y}(t)$ is the output vector, which contains all the measurements of the system at time t . $\mathbf{d}(t)$ is a disturbance vector.

2.3.2 Controllability

A system is said to be *controllable* if it is possible to drive the state of the system from any initial state to any desired final state in finite time, using an appropriate control input. For a LTI system, controllability can be determined by the rank of the controllability matrix $\mathcal{C}$, defined as:

$$\mathcal{C} = [\mathbf{B}, \mathbf{A}\mathbf{B}, \mathbf{A}^2\mathbf{B}, \dots, \mathbf{A}^{n-1}\mathbf{B}]. \quad (2.7)$$

If $\text{rank}(\mathcal{C}) = n$, where n is the dimension of the state vector $\mathbf{x}$, then the system is controllable.

2.3.3 MRAC

Model Reference Adaptive Control (MRAC) is a control strategy that aims to make the output of a system follow the output of a reference model, even in the presence of uncertainties or disturbances. The following definition of MRAC is based on [Cite Robust Adaptive Control book, 6.2.3].

Given the n th order LTI-system in state-space form,

$$\frac{d}{dt}\mathbf{x}(t) = \mathbf{A}\mathbf{x}(t) + \mathbf{B}\mathbf{u}(t), \quad \mathbf{x}(t) \in \mathbb{R}^n, \mathbf{u}(t) \in \mathbb{R}^m, \quad (2.8)$$

and the reference model,

$$\frac{d}{dt}\mathbf{x}_m(t) = \mathbf{A}_m\mathbf{x}_m(t) + \mathbf{B}_m\mathbf{r}(t), \quad \mathbf{x}_m(t) \in \mathbb{R}^n, \mathbf{r}(t) \in \mathbb{R}^m. \quad (2.9)$$

The reference model and input $\mathbf{r}(t)$ are chosen so that $\mathbf{x}_m(t)$ represents the desired trajectory that $\mathbf{x}(t)$ should follow. Crucially, $\mathbf{A}_m$ must be a stable matrix, and $(\mathbf{A}, \mathbf{B})$ must be *controllable* for this to work. If matrices $\mathbf{A}$ and $\mathbf{B}$ are known, then the control input $\mathbf{u}(t)$ can be designed as follows:

$$\mathbf{u}(t) = -\mathbf{K}^*\mathbf{x}(t) + \mathbf{L}^*\mathbf{r}(t), \quad \mathbf{K}^* \in \mathbb{R}^{m \times n}, \mathbf{L}^* \in \mathbb{R}^{m \times m}. \quad (2.10)$$

This obtains the closed loop system:

$$\frac{d}{dt}\mathbf{x}(t) = (\mathbf{A} - \mathbf{B}\mathbf{K}^*)\mathbf{x}(t) + \mathbf{B}\mathbf{L}^*\mathbf{r}(t). \quad (2.11)$$

If we choose $\mathbf{K}^*$ and $\mathbf{L}^*$ such that,

$$\mathbf{A} - \mathbf{B}\mathbf{K}^* = \mathbf{A}_m, \quad \mathbf{B}\mathbf{L}^* = \mathbf{B}_m, \quad (2.12)$$

then the closed loop system becomes:

$$\frac{d}{dt}\mathbf{x}(t) = \mathbf{A}_m\mathbf{x}(t) + \mathbf{B}_m\mathbf{r}(t). \quad (2.13)$$

Hence the closed-loop dynamics of $\mathbf{x}(t)$ is the same to Equation (2.9), and $\mathbf{x}(t) \rightarrow \mathbf{x}_m(t)$ exponentially fast. But for most cases, $\mathbf{A}$ and $\mathbf{B}$ are not known, which means that $\mathbf{K}^*$ and $\mathbf{L}^*$ has to be estimated online. Assuming that $\mathbf{K}^*$ and $\mathbf{L}^*$ exist, we'll change the input in Equation (2.10) to:

$$\mathbf{u}(t) = -\mathbf{K}(t)\mathbf{x}(t) + \mathbf{L}(t)\mathbf{r}(t), \quad \mathbf{K}(t) \in \mathbb{R}^{m \times n}, \mathbf{L}(t) \in \mathbb{R}^{m \times m}. \quad (2.14)$$

Adaptive Law

Defining the tracking error $\mathbf{e}(t)$ as:

$$\mathbf{e}(t) = \mathbf{x}(t) - \mathbf{x}_m(t). \quad (2.15)$$

Solving for $\mathbf{P}$ for a $\mathbf{Q} = \mathbf{Q}^T > 0$ in the Lyapunov equation:

$$\mathbf{A}_m^T \mathbf{P} + \mathbf{P} \mathbf{A}_m = -\mathbf{Q}, \quad (2.16)$$

we have the adaptive law:

$$\begin{aligned}\dot{\mathbf{K}}(t) &= \mathbf{B}_m^T \mathbf{P} \mathbf{e} \mathbf{x}^T \operatorname{sgn}(l), \\ \dot{\mathbf{L}}(t) &= -\mathbf{B}_m^T \mathbf{P} \mathbf{e} \mathbf{r}^T \operatorname{sgn}(l).\end{aligned}\tag{2.17}$$

Chapter 3

Modelling

3.1 Finger model

For modelling the finger, we're assuming that the motion is limited to a 2D-plane. This is mostly true, due to the collateral ligaments at the DIP- and the PIP-joint, preventing excessive side-to-side bending, as discussed in Section 2.1.3. The MCP joint doesn't have the same support-structure, so it's free to move side to side, but this isn't as relevant to model for the task at hand, as the lateral movement from the cable pulling on the finger is minimal.

For our model, we assume that the finger is composed of three links, with three revolute joints, like a *planar manipulator* (**Reference to theory: planar manipulator**). To model the stiffness from the tendons and ligaments, we'll add a torsional spring at each joint. The springs will have a non-zero equilibrium angle, representing the natural resting position of the finger joints. The damping from the soft tissue will be modeled by torsional dampers. A schematic of the model can be seen in Figure 3.1. We'll model the links as rigid cylinders, with uniform mass distribution, and the joints as ideal revolute joints, with no friction. The parameters of the model are listed in Figure 3.2.

To model the dynamics of the finger, we'll use the Euler-Lagrange equations as shown in (**Add reference to theory: Euler-Lagrange Equations**). We'll define our generalized coordinates as the relative joint angles, φ_{MCP} , φ_{PIP} and φ_{DIP} :

$$\mathbf{q} \triangleq \begin{bmatrix} \varphi_{\text{MCP}} \\ \varphi_{\text{PIP}} \\ \varphi_{\text{DIP}} \end{bmatrix} \implies \dot{\mathbf{q}} = \begin{bmatrix} \dot{\varphi}_{\text{MCP}} \\ \dot{\varphi}_{\text{PIP}} \\ \dot{\varphi}_{\text{DIP}} \end{bmatrix} \implies \ddot{\mathbf{q}} = \begin{bmatrix} \ddot{\varphi}_{\text{MCP}} \\ \ddot{\varphi}_{\text{PIP}} \\ \ddot{\varphi}_{\text{DIP}} \end{bmatrix}. \quad (3.1)$$

To derive the equations of motion, we need to define the kinetic energy, T , and the potential energy, V of our system.

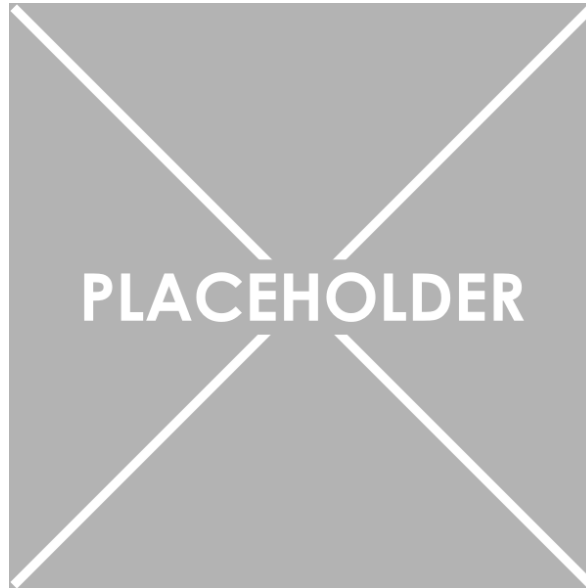


Figure 3.1: Schematic of the finger model, with three revolute joints, torsional springs and dampers.

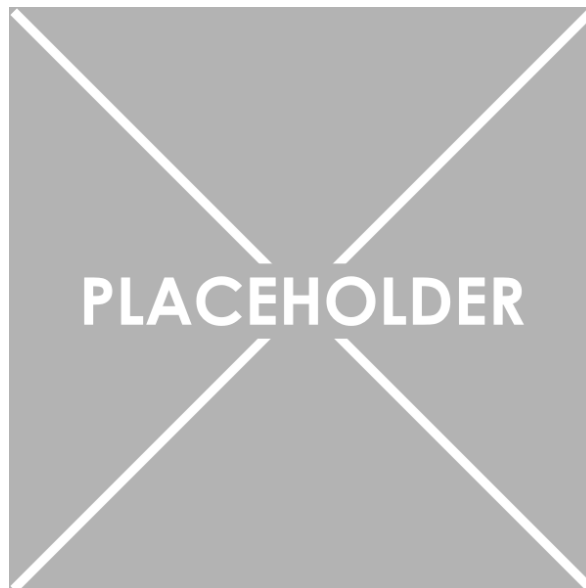


Figure 3.2: Table of model parameters.

3.1.1 Kinetic energy

The kinetic energy will be the sum of the kinetic energy of each link, which can be decomposed into the rotational and translational kinetic energy, defined as:

$$T_{\text{tot}} = T_{\text{rot}} + T_{\text{trans}}. \quad (3.2)$$

Rotational kinetic energy

The rotational kinetic energy of the system is rather straightforward to derive, as the angular velocity of each link is equal to the time derivative of the joint angle. Since the angles are relative, the total angular velocity of each link is the sum of the time derivatives of the joint angles up to that link. Hence, the rotational kinetic energy can be expressed as:

$$T_{\text{rot}} = \frac{1}{2}J_{\text{MCP}}\dot{\varphi}_{\text{MCP}}^2 + \frac{1}{2}J_{\text{PIP}}(\dot{\varphi}_{\text{MCP}} + \dot{\varphi}_{\text{PIP}})^2 + \frac{1}{2}J_{\text{DIP}}(\dot{\varphi}_{\text{MCP}} + \dot{\varphi}_{\text{PIP}} + \dot{\varphi}_{\text{DIP}})^2. \quad (3.3)$$

Translational kinetic energy

The translational kinetic energy is a bit more complex to derive, as the linear velocity of each link depends on the geometry of the system. The linear velocity of each link can be derived by finding the position of the center of mass of each link and taking the time derivative. To find the position of the center of mass, we'll use homogenous transformations, as described in Section 2.2.1.

Let's start by defining frames for moving between the links. We'll define the static world frame as frame i , attached to the MCP joint, where the x-axis is alligned with the metacarpal. The next is frame PP , attached to the PIP joint, where the x-axis is alligned with the proximal phalanx. Subsequently we have frame MP , attached to the DIP joint, where the x-axis is alligned with the middle phalanx. Finally, we have frame DP , attached to the tip of the finger, where the x-axis is alligned with the distal phalanx. The frames are shown in Figure 3.3. The homogenous transformations between these frames can be expressed as:

$$\begin{aligned} \mathbf{H}_{PP}^i &= \begin{bmatrix} \mathbf{R}(\varphi_{\text{MCP}}) & \mathbf{R}(\varphi_{\text{MCP}})\mathbf{e}_1 l_{\text{PP}} \\ \mathbf{0}_{1 \times 2} & 1 \end{bmatrix}, \\ \mathbf{H}_{MP}^{PP} &= \begin{bmatrix} \mathbf{R}(\varphi_{\text{PIP}}) & \mathbf{R}(\varphi_{\text{PIP}})\mathbf{e}_1 l_{\text{MP}} \\ \mathbf{0}_{1 \times 2} & 1 \end{bmatrix}, \\ \mathbf{H}_{DP}^{MP} &= \begin{bmatrix} \mathbf{R}(\varphi_{\text{DIP}}) & \mathbf{R}(\varphi_{\text{DIP}})\mathbf{e}_1 l_{\text{DP}} \\ \mathbf{0}_{1 \times 2} & 1 \end{bmatrix}. \end{aligned} \quad (3.4)$$



Figure 3.3: Schematic of the homogenous transformation frames.

Assuming the the center of mass of each link is located at the midpoint of the link, we can express the position of the center of mass of each link in the world frame as:

$$\begin{aligned}
 \mathbf{p}_{PP,CM}^i &= \mathbf{H}_{PP}^i \begin{bmatrix} -l_{PP}/2 \\ 0 \\ 1 \end{bmatrix}, \\
 \mathbf{p}_{MP,CM}^i &= \mathbf{H}_{PP}^i \mathbf{H}_{MP}^{PP} \begin{bmatrix} -l_{MP}/2 \\ 0 \\ 1 \end{bmatrix}, \\
 \mathbf{p}_{DP,CM}^i &= \mathbf{H}_{PP}^i \mathbf{H}_{MP}^{PP} \mathbf{H}_{DP}^{MP} \begin{bmatrix} -l_{DP}/2 \\ 0 \\ 1 \end{bmatrix}.
 \end{aligned} \tag{3.5}$$

Extracting the x and y components of the position, defining them as $\mathbf{r}_{PP,CM}$, $\mathbf{r}_{MP,CM}$, and $\mathbf{r}_{DP,CM}$, we can take the time derivative to find the linear velocity of each center of mass, which can then be used to calculate the translational kinetic energy as:

$$T_{\text{trans}} = \frac{1}{2} \left[m_{PP} \dot{\mathbf{r}}_{PP,CM}^T \dot{\mathbf{r}}_{PP,CM} + m_{MP} \dot{\mathbf{r}}_{MP,CM}^T \dot{\mathbf{r}}_{MP,CM} + m_{DP} \dot{\mathbf{r}}_{DP,CM}^T \dot{\mathbf{r}}_{DP,CM} \right]. \tag{3.6}$$

3.1.2 Potential energy

For the potential energy one would normally consider the gravitational potential energy. However, since the finger is relatively light and the motion during gripping tasks is mostly in the horizontal plane, we'll omit the gravitational potential energy for simplicity. Also, for clarity we'll model the torsional springs as generalized forces,

instead of including them in the potential energy. Hence, the potential energy of the system can be expressed as:

$$V = 0. \quad (3.7)$$

3.1.3 Generalized forces

The generalized forces in our system will be the torques from the torsional springs and dampers, aswell as the torques from the cable pulling on the finger, and other external torques when the finger interacts with objects. The torques from the torsional springs can be expressed as:

$$\begin{aligned} \boldsymbol{\tau}_{\text{spring}} &= \begin{bmatrix} k_{\text{MCP}}(\varphi_{\text{MCP}} - \varphi_{\text{MCP,eq}}) \\ k_{\text{PIP}}(\varphi_{\text{PIP}} - \varphi_{\text{PIP,eq}}) \\ k_{\text{DIP}}(\varphi_{\text{DIP}} - \varphi_{\text{DIP,eq}}) \end{bmatrix} = \mathbf{K} (\mathbf{q} - \mathbf{q}_{\text{eq}}) \\ \mathbf{K} &\triangleq \begin{bmatrix} k_{\text{MCP}} & 0 & 0 \\ 0 & k_{\text{PIP}} & 0 \\ 0 & 0 & k_{\text{DIP}} \end{bmatrix}, \quad \mathbf{q}_{\text{eq}} \triangleq \begin{bmatrix} \varphi_{\text{MCP,eq}} \\ \varphi_{\text{PIP,eq}} \\ \varphi_{\text{DIP,eq}} \end{bmatrix}. \end{aligned} \quad (3.8)$$

The torques from the torsional dampers can be expressed as:

$$\boldsymbol{\tau}_{\text{damper}} = \begin{bmatrix} b_{\text{MCP}}\dot{\varphi}_{\text{MCP}} \\ b_{\text{PIP}}\dot{\varphi}_{\text{PIP}} \\ b_{\text{DIP}}\dot{\varphi}_{\text{DIP}} \end{bmatrix} = \mathbf{B}\dot{\mathbf{q}}, \quad \mathbf{B} \triangleq \begin{bmatrix} b_{\text{MCP}} & 0 & 0 \\ 0 & b_{\text{PIP}} & 0 \\ 0 & 0 & b_{\text{DIP}} \end{bmatrix}. \quad (3.9)$$

The total generalized forces can then be expressed as:

$$\boldsymbol{\tau} = \boldsymbol{\tau}_{\text{ext}} + \boldsymbol{\tau}_{\text{cable}} - \boldsymbol{\tau}_{\text{spring}} - \boldsymbol{\tau}_{\text{damper}}. \quad (3.10)$$

3.1.4 Rigid-body dynamics

For rigid-body mechanical systems, like the one we're modelling, the Euler-Lagrange equations can be expressed in matrix form as (**Add reference to theory: Rigid-body dynamics**):

$$\mathbf{M}(\mathbf{q})\ddot{\mathbf{q}} + \mathbf{C}(\mathbf{q}, \dot{\mathbf{q}})\dot{\mathbf{q}} + \mathbf{G}(\mathbf{q}) = \boldsymbol{\tau}. \quad (3.11)$$

The *inertia matrix* $\mathbf{M}(\mathbf{q})$ is a symmetric positive-definite matrix that represents the mass distribution of the system. We'll define $\varphi_{\text{MCP}} \triangleq \varphi_1$, $\varphi_{\text{PIP}} \triangleq \varphi_2$, and $\varphi_{\text{DIP}} \triangleq \varphi_3$, the cumulative angles $\varphi_{12} \triangleq \varphi_1 + \varphi_2$, $\varphi_{23} \triangleq \varphi_2 + \varphi_3$, and the shorthand form $c_\varphi \triangleq \cos(\varphi)$, $s_\varphi \triangleq \sin(\varphi)$. Based on the E.L.-equations the inertia matrix in our system then equates to:

$$\mathbf{M}(\mathbf{q}) = \begin{bmatrix} p_1 + p_2 c_{\varphi_2} + p_3 c_{\varphi_{23}} + p_4 c_{\varphi_3} & p_4 c_{\varphi_3} + p_5 + p_6 c_{\varphi_2} + p_7 c_{\varphi_{23}} & p_7 c_{\varphi_{23}} + p_8 + p_9 c_{\varphi_3} \\ p_4 c_{\varphi_3} + p_5 + p_6 c_{\varphi_2} + p_7 c_{\varphi_{23}} & p_4 c_{\varphi_3} + p_5 & p_8 + p_9 c_{\varphi_3} \\ p_7 c_{\varphi_{23}} + p_8 + p_9 c_{\varphi_3} & p_8 + p_9 c_{\varphi_3} & p_8 \end{bmatrix}, \quad (3.12)$$

where the different p_i 's are shown in Table 3.1. The *coriolis and centrifugal matrix* $\mathbf{C}(\mathbf{q}, \dot{\mathbf{q}})$ represents the coriolis and centrifugal forces that arise from the motion of the system. For our system, it equates to:

$$\mathbf{C}(\mathbf{q}, \dot{\mathbf{q}}) = \frac{1}{2} \begin{bmatrix} \gamma_{11} & \gamma_{12} & \gamma_{13} \\ \gamma_{21} & \gamma_{22} & \gamma_{23} \\ \gamma_{31} & \gamma_{32} & 0 \end{bmatrix}, \quad (3.13)$$

where

$$\begin{aligned} \gamma_{11} &= -p_2 \dot{\varphi}_2 s_{\varphi_2} - p_3 \dot{\varphi}_{2-3} s_{\varphi_{23}} - p_4 \dot{\varphi}_3 s_{\varphi_3}, \\ \gamma_{12} &= -(p_2 \dot{\varphi}_1 + 2p_6 \dot{\varphi}_2) s_{\varphi_2} - (p_3 \dot{\varphi}_1 + 2p_7 \dot{\varphi}_{2-3}) s_{\varphi_{23}} - p_4 \dot{\varphi}_3 s_{\varphi_3}, \\ \gamma_{13} &= -(p_3 \dot{\varphi}_1 + 2p_7 \dot{\varphi}_{2-3}) s_{\varphi_{23}} - (p_4 \dot{\varphi}_{1-2} + 2p_9 \dot{\varphi}_3) s_{\varphi_3}, \\ \gamma_{21} &= \dot{\varphi}_1 (p_2 s_{\varphi_2} + p_3 s_{\varphi_{23}}) - \dot{\varphi}_3 p_4 s_{\varphi_3}, \\ \gamma_{22} &= -\dot{\varphi}_3 p_4 s_{\varphi_3}, \\ \gamma_{23} &= -(p_4 \dot{\varphi}_{1-2} + 2p_9 \dot{\varphi}_3) s_{\varphi_3}, \\ \gamma_{31} &= \dot{\varphi}_1 p_3 s_{\varphi_{23}} + p_4 \dot{\varphi}_{1-2} s_{\varphi_3}, \\ \gamma_{32} &= p_4 \dot{\varphi}_{1-2} s_{\varphi_3}. \end{aligned}$$

The generalized forces $\boldsymbol{\tau}$ are the same as in Equation (3.10). The *gravity vector* $\mathbf{G}(\mathbf{q})$ represents the gravitational forces acting on the system. Since we're omitting the effects of gravity, the gravity vector is zero:

$$\mathbf{G}(\mathbf{q}) = \mathbf{0}. \quad (3.14)$$

Inserting all the results above, and solving for $\ddot{\mathbf{q}}$, we can express the equations of motion of our system as:

$$\ddot{\mathbf{q}} = \mathbf{M}(\mathbf{q})^{-1} [\boldsymbol{\tau}_{\text{ext}} + \boldsymbol{\tau}_{\text{cable}} - (\mathbf{C}(\mathbf{q}, \dot{\mathbf{q}}) + \mathbf{B}) \dot{\mathbf{q}} - \mathbf{K}(\mathbf{q} - \mathbf{q}_{\text{eq}})]. \quad (3.15)$$

Table 3.1: Constants in system matrices.

Constant	Value
p_1	$J_{MCP} + J_{PIP} + J_{DIP} + \frac{l_{PP}^2}{4}m_{PP} + \left(l_{PP}^2 + \frac{l_{MP}^2}{4}\right)m_{MP} + \left(l_{PP}^2 + l_{MP}^2 + \frac{l_{DP}^2}{4}\right)m_{DP}$
p_2	$l_{PP}l_{MP}(m_{MP} + 2m_{DP})$
p_3	$l_{PP}l_{DP}m_{DP}$
p_4	$l_{MP}l_{DP}m_{DP}$
p_5	$J_{PIP} + J_{DIP} + \frac{l_{MP}^2}{4}m_{MP} + \left(l_{MP}^2 + \frac{l_{DP}^2}{4}\right)m_{DP}$
p_6	$\frac{1}{2}l_{PP}l_{MP}(m_{MP} + 2m_{DP})$
p_7	$\frac{1}{2}l_{PP}l_{DP}m_{DP}$
p_8	$J_{DIP} + \frac{l_{DP}^2}{4}m_{DP}$
p_9	$\frac{1}{2}l_{MP}l_{DP}m_{DP}$

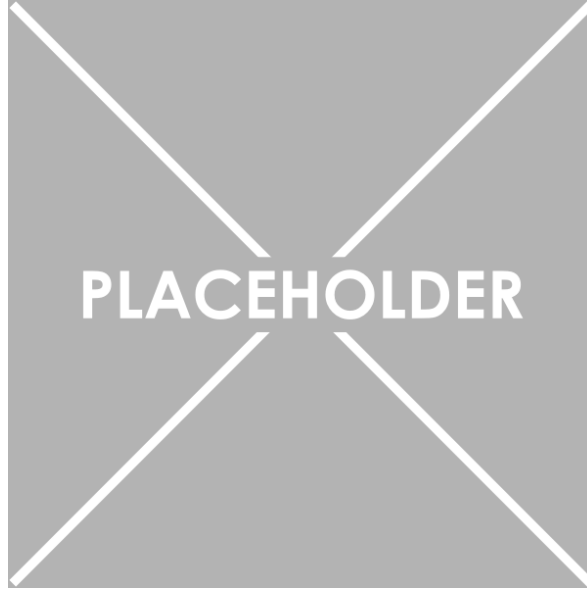


Figure 3.4: Circuit diagram of a brushed DC motor.

3.2 Brushed DC Motor model

In our system we're using a 12 V Brushed DC motor from Pololu [**Cite to motor**]. I've based my dynamical model from [**Cite to article describing Brushed DC Motor**]. The circuit diagram of a brushed DC motor is shown in Section 3.2. Equation (3.16) describes the mechanical dynamics of the motor, and Equation (3.17) describes the electrical dynamics of the motor, defined as:

$$J_m \frac{d^2\theta}{dt^2} + B_m \frac{d\theta}{dt} = \tau_m - \tau_l \quad (3.16)$$

$$R_a I_a + L_a \frac{dI_a}{dt} + E_b = E_a. \quad (3.17)$$

Here, θ defines the angle of the motor. The torque generated by the motor is given by $\tau_m = K_t I_a$, where K_t is the torque constant of the motor. The load torque τ_l represents the torque exerted by the finger on the motor through the cable, as well as friction between the motor and the finger. The friction within the rotor of the motor is modeled with $B_m \frac{d\theta}{dt}$, where B_m is a constant friction coefficient. The inertia of the rotor is given by J_m . The back electromotive force (EMF) is given by $E_b = K_b \frac{d\theta}{dt}$, where K_b is the back EMF constant of the motor. The armature voltage E_a is the control input to the system. R_a and L_a are the armature resistance and inductance, respectively. With the change to τ_m and E_b , we can rewrite the equations as:

$$J_m \frac{d^2\theta}{dt^2} + B_m \frac{d\theta}{dt} = K_t I_a - \tau_l \quad (3.18)$$

$$R_a I_a + L_a \frac{dI_a}{dt} + K_b \frac{d\theta}{dt} = E_a. \quad (3.19)$$

3.2.1 Estimating motor parameters

To estimate the parameters of the motor, we will use the datasheet from the manufacturer [Cite motor datasheet]. Relevant information extracted from the datasheet is shown in Table 3.2. Considering the motor at stall, we have $I_a = I_{a, \text{stall}}$ and $\tau_l = \tau_{l, \text{stall}}$. In addition we have $\frac{d\theta}{dt} \equiv 0$, and at stationarity, $\frac{dI_a}{dt} \equiv 0$. This allows us to rewrite Equation (3.19) as:

$$R_a I_{a, \text{stall}} = E_a \implies R_a = \frac{E_a}{I_{a, \text{stall}}} \approx 2.44 \Omega. \quad (3.20)$$

Further, when the motor is stalled, we should expect the torque generated by the motor to equal the stall torque, giving us:

$$K_t I_{a, \text{stall}} = \tau_{l, \text{stall}} \implies K_t = \frac{\tau_{l, \text{stall}}}{I_{a, \text{stall}}} \approx 0.44 \text{ N m A}^{-1}. \quad (3.21)$$

For the case when the motor is running at no-load speed, we have $I_a = I_{a, \text{nl}}$ and $\frac{d\theta}{dt} = \omega_{nl}$, and at stationarity, $\frac{dI_a}{dt} \equiv 0$. This allows us to rewrite Equation (3.19) as:

$$R_a I_{a, \text{nl}} + K_b \omega_{nl} = E_a \implies K_b = \frac{E_a - R_a I_{a, \text{nl}}}{\omega_{nl}} \approx 0.845 \text{ V s rad}^{-1}. \quad (3.22)$$

Further, at no-load speed, we have $\tau_l \equiv 0$, and $\frac{d^2\theta}{dt^2} \equiv 0$. This allows us to rewrite Equation (3.18) as:

$$B_m \omega_{nl} = K_t I_{a, \text{nl}} \implies B_m = \frac{K_t I_{a, \text{nl}}}{\omega_{nl}} \approx 6.47 \text{ mN m s rad}^{-1}. \quad (3.23)$$

Sadly it's not possible to estimate the inertia of the rotor J_m , nor the armature

Table 3.2: Relevant motor parameters extracted from the datasheet of the motor.

Parameter	Symbol	Value
Armature Voltage	E_a	12 V
Stall Torque	$\tau_{l, stall}$	2.158 N m
Stall Current	$I_{a, stall}$	4.9 A
No-load Speed	ω_{nl}	13.61 rad/sec
No-load Current	$I_{a, nl}$	0.20 A

inductance L_a from the datasheet. For simulation purposes, the parameters are chosen as $J_m = 0.093 \text{ kg m}^2$ and $L_a = 6 \text{ mH}$, reported as example brushed DC motor values by Rahman and Mat Yahya (2021). Even though the values are currently unknown, they can be estimated through a calibration step later. One way that's possibly the most convenient, is to use a parameter identification method.

3.2.2 Estimating the armature inductance and rotor inertia

To estimate the armature inductance and rotor inertia, we can use a parameter identification method. We'll start by converting Equation (3.19) to the Laplace domain, giving us:

$$E_a(s) = (L_a s + R_a)I_a(s) + K_b s\theta(s). \quad (3.24)$$

Since we only measure $I_a(t)$ and not its derivative, we have to apply a proper filter of order 1, defined as:

$$\Lambda_{I_a}(s) = s + \lambda_0, \quad \lambda_0 > 0. \quad (3.25)$$

Applying the filter to Equation (3.24) gives us:

$$\frac{1}{\Lambda_{I_a}(s)}E_a(s) = \frac{1}{\Lambda_{I_a}(s)}(L_a s + R_a)I_a(s) + \frac{1}{\Lambda_{I_a}(s)}K_b s\theta(s).$$

Ordering it around, moving all measured signals and known parameters to the left, and all factors with the unknown parameter L_a to the right, we get:

$$\underbrace{\frac{1}{\Lambda_{I_a}(s)}[E_a(s)] - \frac{1}{\Lambda_{I_a}(s)}[R_a I_a(s)] - \frac{s}{\Lambda_{I_a}(s)}[K_b \theta(s)]}_{z_{L_a}} = \underbrace{\frac{s}{\Lambda_{I_a}(s)}[I_a(s)]}_{\Phi_{L_a}} \underbrace{L_a}_{\theta_{L_a}^*} \quad (3.26)$$

$$\implies z_{L_a} = \theta_{L_a}^* \Phi_{L_a}.$$

Hence we've transformed the system to a SPM like discussed in [Add reference

to **Theory:SPM**]. We can do the same for the inertia of the rotor J_m . We'll start by converting Equation (3.18) to the Laplace domain, assuming there is no external load ($\tau_l \equiv 0$), giving us:

$$(J_m s^2 + B_m s) \theta(s) = K_t I_a(s). \quad (3.27)$$

Since we're only measuring $\theta(t)$ and not its derivatives, we have to apply a proper filter of order 2, defined as:

$$\Lambda_\theta(s) = s^2 + \lambda_1 s + \lambda_0, \quad \lambda_0, \lambda_1 > 0. \quad (3.28)$$

Applying the filter to Equation (3.27), moving all measured signals and known parameters to the left, and all factors with the unknown parameter J_m to the right, we get:

$$\underbrace{\frac{1}{\Lambda_\theta(s)} [K_t I_a(s)] - \frac{s}{\Lambda_\theta(s)} [B_m \theta(s)]}_{z_{J_m}} = \underbrace{\frac{s^2}{\Lambda_\theta(s)} [\theta(s)]}_{\Phi_{J_m}} \underbrace{J_m}_{\theta_{J_m}^*}. \quad (3.29)$$

$$\implies z_{J_m} = \theta_{J_m}^* \Phi_{J_m}.$$

Combining Equation (3.26) and Equation (3.29), we can express them both in SPM form as:

$$\underbrace{\begin{bmatrix} z_{L_a} \\ z_{J_m} \end{bmatrix}}_{\mathbf{z}} = \underbrace{\begin{bmatrix} L_a & J_m \end{bmatrix}}_{\boldsymbol{\theta}^{*\top}} \underbrace{\begin{bmatrix} \Phi_{L_a} \\ \Phi_{J_m} \end{bmatrix}}_{\boldsymbol{\Phi}}. \quad (3.30)$$

$$\implies \mathbf{z} = \boldsymbol{\theta}^{*\top} \boldsymbol{\Phi}.$$

Using the gradient method developed in [**Add reference to Theory: Gradient method**], with

$$\lambda_{0,L_a} = 10, \quad \lambda_{0,J_m} = 0.1, \quad \lambda_{1,J_m} = 0.1, \quad \boldsymbol{\Gamma} = \begin{bmatrix} 10 & 0 \\ 0 & 1000 \end{bmatrix},$$

$$\hat{\boldsymbol{\theta}}_0 = \begin{bmatrix} \hat{L}_a(0) \\ \hat{J}_m(0) \end{bmatrix} = \begin{bmatrix} 1 \text{ mH} \\ 0.01 \text{ kg m}^2 \end{bmatrix}, \quad \boldsymbol{\theta}^* = \begin{bmatrix} L_a \\ J_m \end{bmatrix} = \begin{bmatrix} 6 \text{ mH} \\ 0.093 \text{ kg m}^2 \end{bmatrix},$$

while applying a step input $E_a = 12 \text{ V}$ at $t = 50 \text{ ms}$, the resulting estimates of $\hat{L}_a(t)$ and $\hat{J}_m(t)$ are shown in Section 3.2.2. It is clear that the estimates converge exponentially fast to the true values. Since these are static parameters that only needs to be estimated once, L_a and J_m will be assumed known for further controller design.

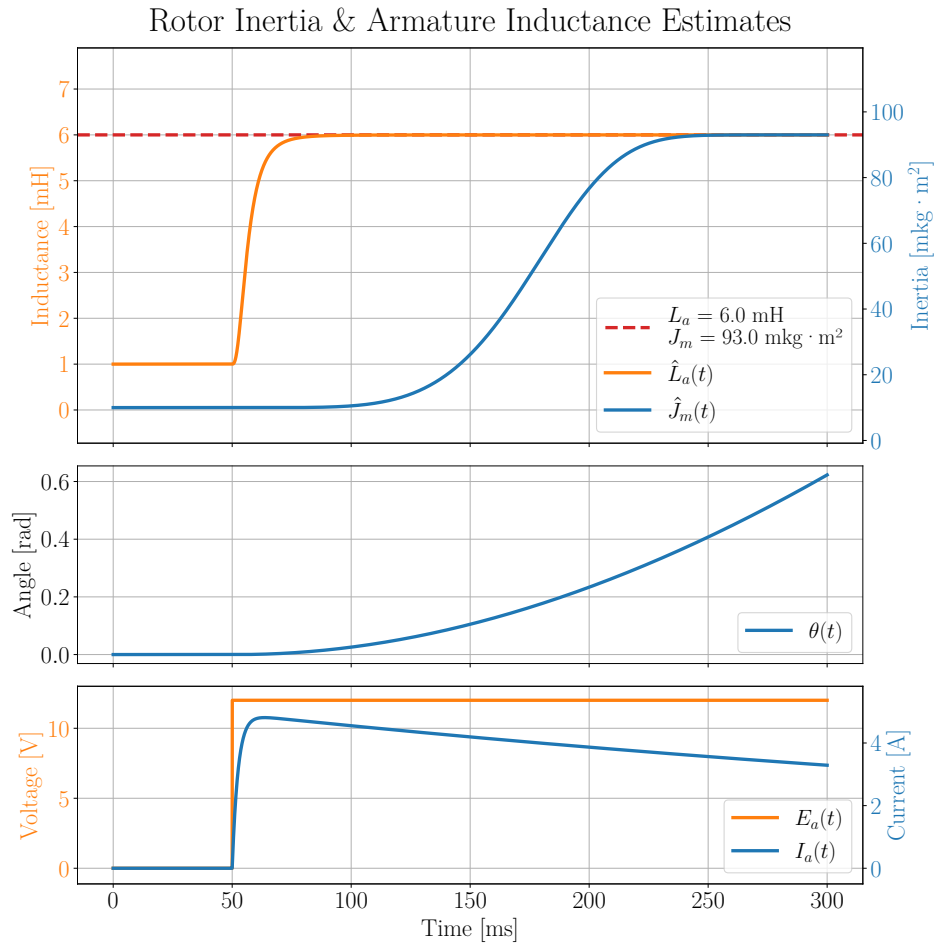


Figure 3.5: Parameter identification of the armature inductance L_a and rotor inertia J_m using a step input in the armature voltage E_a .

3.2.3 Mass-spring-damper load

In Section 3.1 we assumed that the finger could be modelled as several torsional mass-spring-damper systems. Since we can't measure the finger-angles, in addition to that the configuration of the whiffle tree is unknown, the full finger model isn't observable, and can't be used for position control design. Instead, we'll assume that finger can be modelled as a single mass-spring-damper system, acting on the motor as τ_l . This is a simplification, but it allows for a better analysis of the system. Hence we'll model the load torque τ_l as:

$$\tau_l = J \frac{d^2\theta}{dt^2} + c \frac{d\theta}{dt} + k(\theta - \theta_{eq}), \quad (3.31)$$

where J is the 'total' inertia of the finger, c is the 'total' damping coefficient of the finger, k is the 'total' stiffness of the finger, and θ_{eq} is a kind of equilibrium angle of the finger. Substituting this into Equation (3.18), we get the updated mechanical dynamics:

$$(J_m + J) \frac{d^2\theta}{dt^2} + (B_m + c) \frac{d\theta}{dt} + k\theta = K_t I_a + k\theta_{eq} \quad (3.32)$$

3.2.4 State-space model of brushed DC motor with load

To analyze the position controller for the motor with load, discussed further in Section 4.3, we need to express the dynamics in Equation (3.32) in state-space representation, as shown in Section 2.3.1. Assuming that I_a can be treated as an input, explained in Section 4.1, and defining $\omega \triangleq \frac{d\theta}{dt}$, we can rewrite the Equation (3.32) and as:

$$\frac{d\omega}{dt} = \frac{1}{J_m + J} (K_t I_a - (B_m + c)\omega - k(\theta - \theta_{eq})). \quad (3.33)$$

Written in state-space form, we have:

$$\begin{aligned} \underbrace{\frac{d}{dt} \begin{bmatrix} \theta \\ \omega \end{bmatrix}}_{\frac{d}{dt} \mathbf{x}} &= \underbrace{\begin{bmatrix} 0 & 1 \\ -\frac{k}{J_m+J} & -\frac{B_m+c}{J_m+J} \end{bmatrix}}_{\mathbf{A}} \underbrace{\begin{bmatrix} \theta \\ \omega \end{bmatrix}}_{\mathbf{x}} + \underbrace{\begin{bmatrix} 0 \\ \frac{K_t}{J_m+J} \end{bmatrix}}_{\mathbf{B}} \underbrace{I_a}_{\mathbf{u}} + \underbrace{\begin{bmatrix} 0 \\ \frac{k\theta_{eq}}{J_m+J} \end{bmatrix}}_{\mathbf{d}} \\ &\implies \frac{d}{dt} \mathbf{x} = \mathbf{Ax} + \mathbf{Bu} + \mathbf{d}. \end{aligned} \quad (3.34)$$

In addition we have an encoder measuring θ . Hence the output of the system is given by:

$$\begin{aligned} \underbrace{\begin{bmatrix} \theta \end{bmatrix}}_{\mathbf{y}} &= \underbrace{\begin{bmatrix} 1 & 0 \end{bmatrix}}_{\mathbf{C}} \underbrace{\begin{bmatrix} \theta \\ \omega \end{bmatrix}}_{\mathbf{x}} \\ &\implies \mathbf{y} = \mathbf{Cx}. \end{aligned} \quad (3.35)$$

Chapter 4

Controller design

4.1 Current controller

To help facilitate the design of the position and torque controllers, we will first design a controller for the armature current I_a with the armature voltage E_a . This will allow us to treat the armature current as a control input in the position control design, simplifying the dynamics significantly. As the dynamics of the armature current is so fast, the controller has to be as cheap as possible to compute. Hence, it will be designed as a simple proportional (P) controller, but with a feedforward (FF) term to account for the back EMF. Let's first consider the electrical dynamics of the motor from Equation (3.19):

$$L_a \frac{dI_a}{dt} + R_a I_a + K_b \frac{d\theta}{dt} = E_a. \quad (4.1)$$

Given the time varying reference current $r(t)$, we can design the input voltage $E_a(t)$ as:

$$E_a(t) = \underbrace{K_b \frac{d\theta(t)}{dt}}_{\text{Feedforward term}} + \underbrace{L_a K_p (r(t) - I_a(t))}_{\text{P controller}}. \quad (4.2)$$

Substituting Equation (4.2) into Equation (4.1), we get:

$$\frac{dI_a(t)}{dt} + R_a I_a(t) = K_p (r(t) - I_a(t)) \triangleq u(t). \quad (4.3)$$

This controller is supposed to be ran in real time on a microcontroller. That way it won't have access to continuous time signals, but rather discrete time signals. Hence, we will need to discretize the controller for stability analysis and tuning. For this analysis we will assume that the dynamics of the motor are slow enough such that the feedforward term can be treated as a constant. This is a reasonable assumption since the feedforward term is based on the angular velocity, which changes relatively slowly

compared to the armature current.

4.1.1 Discretization of current controller

Suppose the controller is ran with a update period of T_s . Assuming that the input voltage E_a is held constant between sampling times, i.e.,

$$u(t) = u[k] = K_p(r[k] - I_a[k]), \quad kT_s \leq t < (k+1)T_s, \quad (4.4)$$

the dynamics in Equation (4.3) then becomes:

$$\frac{dI_a(\tau)}{d\tau} + R_a I_a(\tau) = u[k], \quad \tau = t - kT_s \in [0, T_s]. \quad (4.5)$$

To solve it, let's multiply by the integrating factor $e^{R_a\tau}$, which gives:

$$e^{R_a\tau} \left[\frac{dI_a(\tau)}{d\tau} + R_a I_a(\tau) \right] = u[k] e^{R_a\tau}. \quad (4.6)$$

Notice how this can be rewritten as:

$$\frac{d}{d\tau} \left(e^{R_a\tau} I_a(\tau) \right) = u[k] e^{R_a\tau}. \quad (4.7)$$

Integrating both sides of Equation (4.7) with respect to τ , over the entire interval from 0 to T_s , we get the discretized dynamics of the current controller:

$$\begin{aligned} [e^{R_a\tau} I_a(\tau)]_{\tau=0}^{\tau=T_s} &= u[k] \int_0^{T_s} e^{R_a\tau} d\tau, \\ \implies e^{R_a T_s} \underbrace{I_a(T_s)}_{I_a[k+1]} - \underbrace{I_a(0)}_{I_a[k]} &= u[k] \frac{e^{R_a T_s} - 1}{R_a}, \\ \implies I_a[k+1] &= \underbrace{e^{-R_a T_s} I_a[k]}_{\triangleq a} + u[k] \underbrace{\frac{1 - e^{-R_a T_s}}{R_a}}_{\triangleq b}, \\ \implies I_a[k+1] &= a I_a[k] + b u[k]. \end{aligned} \quad (4.8)$$

Substituting in $u[k]$ from Equation (4.4), we get:

$$\begin{aligned} I_a[k+1] &= a I_a[k] + b K_p(r[k] - I_a[k]), \\ \implies I_a[k+1] &= (a - b K_p) I_a[k] + b K_p r[k]. \end{aligned} \quad (4.9)$$

4.1.2 Stability analysis and tuning of controller

To guarantee stability of the current controller, we can look at the z-transform of the system. Taking the z-transform of Equation (4.9), disregarding any initial conditions, we get:

$$\begin{aligned}
[z - (a - bK_p)] I_a(z) &= bK_p r(z), \\
\implies \frac{I_a(z)}{r(z)} &= \frac{bK_p}{z - (a - bK_p)}.
\end{aligned} \tag{4.10}$$

Clearly, Equation (4.10) has a pole at $z = a - bK_p$. As discussed in [ADD REFERENCE TO THEORY: DISCRETIZED SYSTEMS], for the system to be stable, we need the pole to be inside the unit circle. In other words, we need:

$$\begin{aligned}
|a - bK_p| < 1 &\implies \left| e^{-R_a T_s} - K_p \frac{1 - e^{-R_a T_s}}{R_a} \right| < 1, \\
\implies K_p &< R_a \frac{1 + e^{-R_a T_s}}{1 - e^{-R_a T_s}} \triangleq K_{p,\max}(T_s).
\end{aligned} \tag{4.11}$$

One downside to using a P controller is that there will be a steady-state error. To eliminate this, we can artificially give the controller a higher reference current. To do this we will need to know the steady-state value of the current for a given reference. Assuming we apply a step reference of $r[k] = r_0$ for all $k \geq 0$. Provided we choose a K_p that satisfies Equation (4.11), the recursion will converge, and as we let $t \rightarrow \infty \implies k \rightarrow \infty$, Equation (4.9) becomes:

$$\begin{aligned}
I_a[\infty] &= (a - bK_p)I_a[\infty] + bK_p r_0, \\
\implies I_a[\infty] &= r_0 \frac{bK_p}{1 - a + bK_p} = r_0 \frac{\frac{1-a}{R_a} K_p}{1 - a + \frac{1-a}{R_a} K_p} = r_0 \frac{K_p}{R_a + K_p}.
\end{aligned} \tag{4.12}$$

Hence to eliminate the steady-state error, we can choose the reference to be:

$$r_1 = \frac{R_a + K_p}{K_p} r_0 \implies I_a[\infty] = r_1 \frac{K_p}{R_a + K_p} = r_0. \tag{4.13}$$

4.1.3 Simulation of current controller

To evaluate the controller, we will simulate it applied to the motor model developed in Section 3.2. Using $K_p = \frac{1}{2}K_{p,\max}(T_s)$, where the reference current is a step input of 1 A, and scaled with regards to Equation (4.13). Additionally there is a saturation put on $E_a = \pm 12$ V. Figure 4.1 and Figure 4.2 shows a simulation of the P+FF controller aswell as just the P controller, with $T_s = 0.2$ ms, where the motor is initially spinning at $\omega_0 = 5$ rad s⁻¹ and $\omega_0 = 0$ rad s⁻¹, respectively. Figure 4.3 and Figure 4.4 shows the simulation of the P+FF controller with and without noise to the measurement of I_a , with $T_s = 0.2$ ms and $T_s = 1.0$ ms, respectively. The noise is a zero-mean Gaussian noise with a standard deviation of $\sigma_{I_a} = 50$ mA, which is higher than is expected for the system, but it will help us evaluate the robustness of the controller.

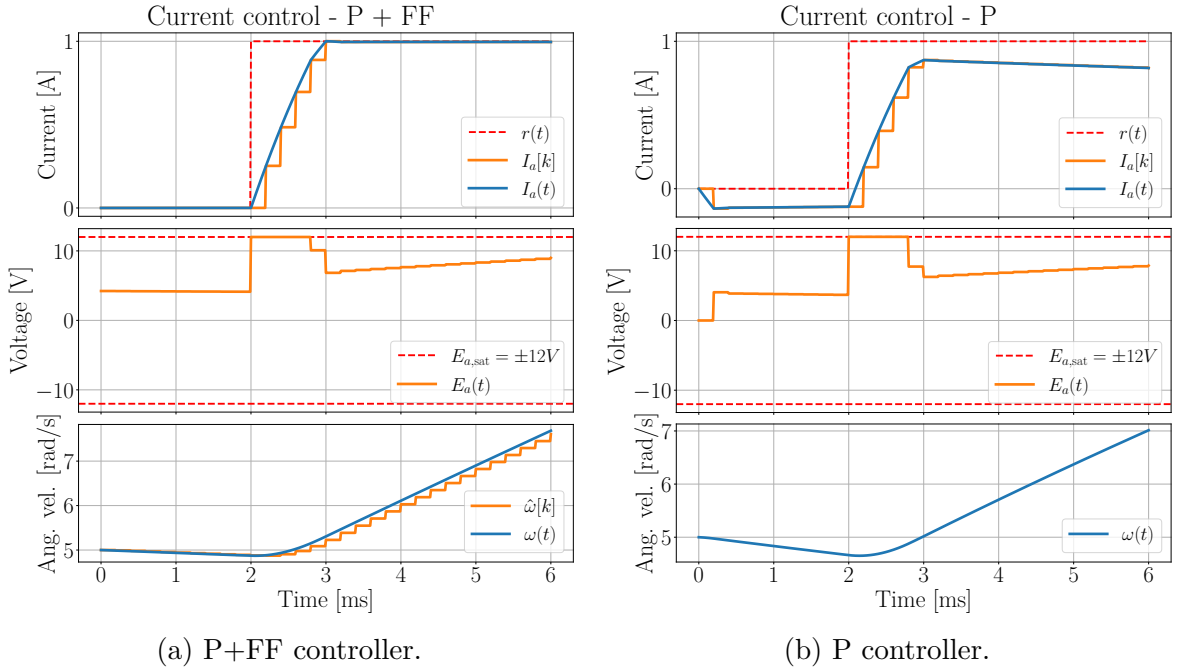


Figure 4.1: Armature current step response for discrete-time P+FF and P controllers tracking a 1 A reference, with $T_s = 0.2$ ms and initial motor speed $\omega_0 = 5$ rad s⁻¹.

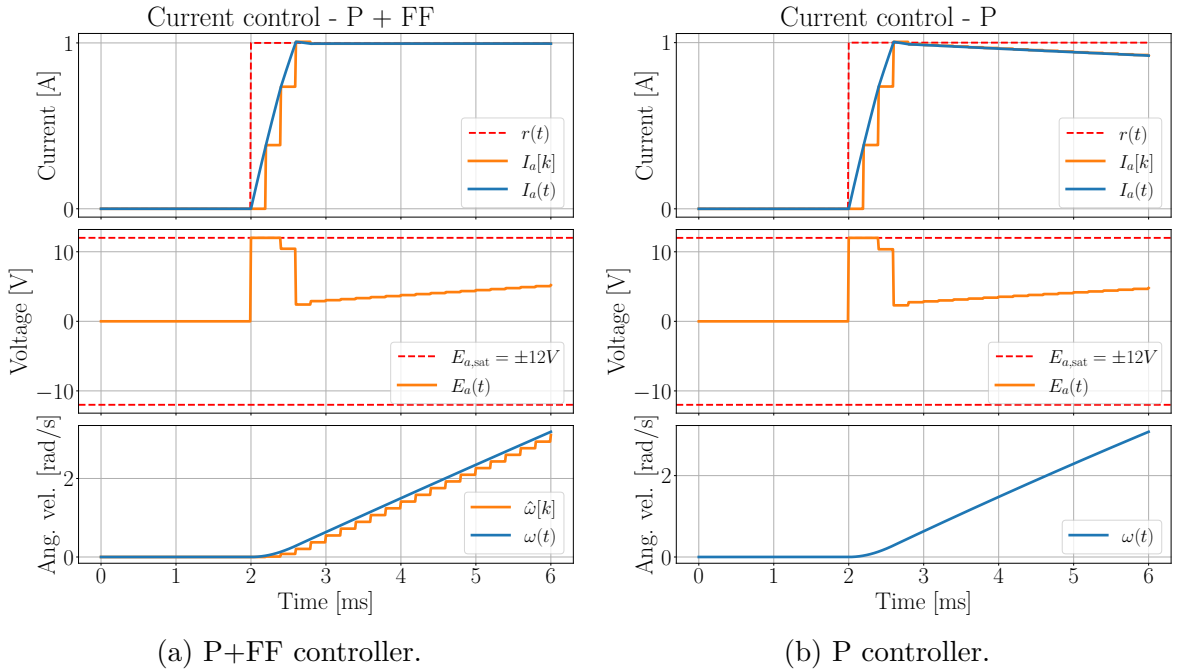


Figure 4.2: Armature current step response for discrete-time P+FF and P controllers tracking a 1 A reference, with $T_s = 0.2$ ms and initial motor speed $\omega_0 = 0$ rad s⁻¹.

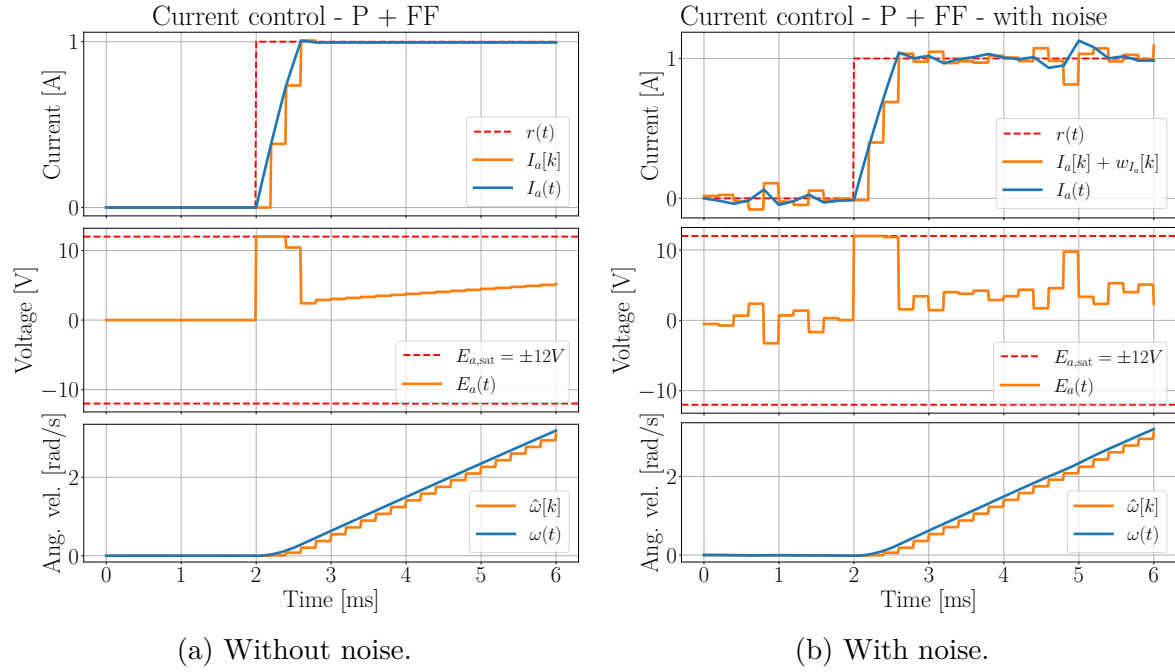


Figure 4.3: Effect of current measurement noise on the armature current response using a discrete-time P+FF controller with $T_s = 0.2$ ms.

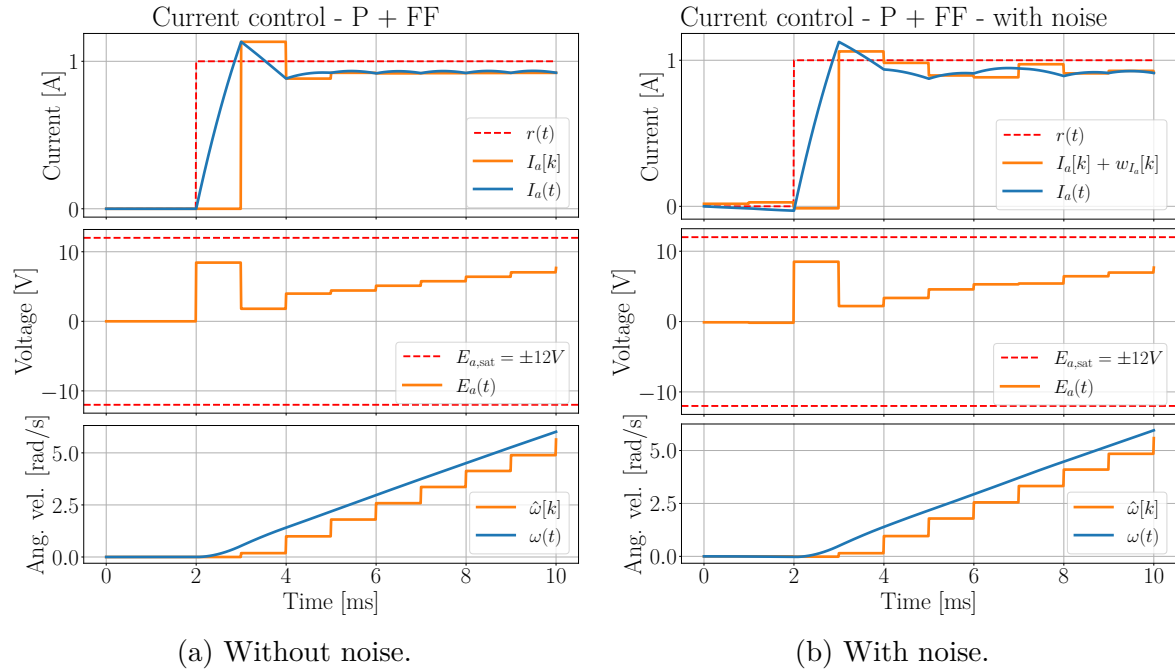


Figure 4.4: Effect of current measurement noise on the armature current response using a discrete-time P+FF controller with $T_s = 1.0$ ms.

4.1.4 Evaluation of controller

When the motor is starting from still, both the P+FF and P controllers are able to reach the target current fast. But as the motor picks up speed, the back-EMF becomes significant, and the P controller struggles to keep up, and the current starts to deviate from the reference current, as can be seen in Figure 4.2b. The P+FF controller on the other hand continues to perform well, as it can account for the back-EMF, as shown in Figure 4.2a. When the motor is already spinning at a high speeds, the differences between the two controllers becomes even more significant. At no point does the P controller reach its target current. In fact, it deviates more and more over time, as can be seen in Figure 4.1b. The P+FF controller on the other hand continues to perform well, as it can account for the back-EMF, as shown in Figure 4.1a.

As can be seen in Figure 4.3 and Figure 4.4, the added noise to the current measurements doesn't really affect the stability of the controller. The current oscillates slightly when there's noise present, compared to the noise-free. There is however a clear difference between longer and shorter update periods. With a longer update period of $T_s = 1.0\text{ ms}$, the current gets a slight steady-state-error, which is not present with the shorter update period of $T_s = 0.2\text{ ms}$. The reason for this is that the assumption of a constant speed between updates is less valid with a longer update period, which causes the feedforward term to be less accurate, which in turn causes the steady-state error. With a shorter update period, the assumption of a constant speed is more valid, which allows the feedforward term to be more accurate, which in turn allows the controller to track the reference current better.

Overall, the P+FF controller shows promising results for controlling the armature current in our system. Due to the saturation in the voltage of $E_a = \pm 12\text{ V}$, there is a maximum rate of change for the current. During simulation it was found to be $\dot{I}_{a_{\max}} \approx 2.0\text{ kA s}^{-1}$, when the motor was starting from still, which is more or less instantaneous seen from the dynamics of the mechanical system. Hence, we will treat the armature current as a control input for further control design.

4.2 Torque control

When the hand interacts with objects, trying to control the position of the hand becomes irrelevant, as the hand will be constrained by the object. Instead, we want to control the torque generated by the motor, which in turn will allow us to control the force applied to the object.

When grasping an object, assuming the object is rigid and that the cable is inelastic, the angular velocity and acceleration of the motor will be $\frac{d\theta}{dt} \equiv 0$ and $\frac{d^2\theta}{dt^2} \equiv 0$, respectively. Based on Equation (3.18), this results in the following relation:

$$\tau_m = K_t I_a = \tau_l. \quad (4.14)$$

Hence, all the torque generated by the motor, which is proportional to the current, would be applied to the load. This means the applied torque can be controlled directly through the current. In Section 4.1 we designed a controller for the armature current I_a through the input armature voltage E_a . With $\frac{d\theta}{dt} \equiv 0$, the back-EMF will be $K_b \frac{d\theta}{dt} \equiv 0$. Therefore we can also simplify the current controller in Equation (4.2) to just a P controller, as there is no back-EMF to account for. This will reduce the computational load on the microcontroller. For a given reference torque r_{τ_m} , we can calculate the reference current r_{I_a} as:

$$r_{I_a} = \frac{r_{\tau_m}}{K_t}. \quad (4.15)$$

If it's more convenient to control the force applied to the object instead of the torque, we can easily convert between the two using the relation $\tau_m = F_m r_{\text{spindle}}$, where r_{spindle} is the radius of the spindle mounted on the motor. Hence, for a given reference force r_F , we can calculate the reference current r_{I_a} as:

$$r_{I_a} = r_F \frac{r_{\text{spindle}}}{K_t}. \quad (4.16)$$

Figure 4.5 shows emulated scenarios of the finger grasping a rigid object, assuming no elasticity in the cable, and no friction between the motor and the finger. The motor is supplied with 3 V for the first 5 ms. Then at $t = 5$ ms, the finger makes contact with the object, and $\frac{d\theta}{dt} = \frac{d^2\theta}{dt^2} \equiv 0$. At the same time, the torque controller is activated, and the reference torque is set to $r_{\tau_m} = 1.5$ N m. For the current controller, the proportional gain is set to $K_p = \frac{1}{2} K_{p,max}$, like in Section 4.1.3. Figure 4.5a and Figure 4.5b show the results for $T_s = 0.2$ ms and $T_s = 1.0$ ms, respectively. As can be seen in the figures, the motor torque τ_m quickly converges to the reference torque r_{τ_m} for both cases.

Ideally, if there was no friction between the motor and the finger, $\tau_l = \tau_{\text{finger}}$. In reality, because of the bowden tube the cable is running through, and other inconsistencies in the setup, there will be some friction. Therefore, we can expect $\tau_l = \tau_{\text{finger}} + \tau_{\text{friction}}$, where τ_{friction} is the torque lost to friction. Hence, without any direct measurement of the applied force to the finger, we would expect that the applied force to the finger will be less than the reference force. However, with integrated force sensors in the fingertip, this wouldn't be an issue.

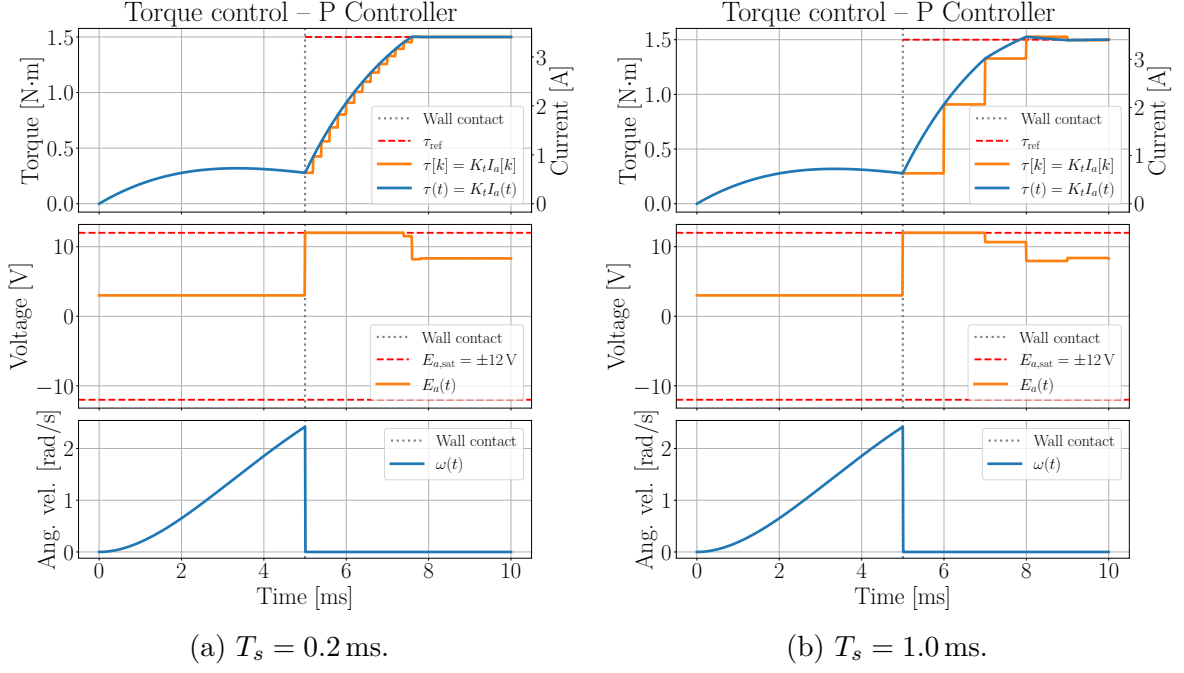


Figure 4.5: Torque control with a P controller when gripping a rigid object.

4.2.1 Force control with integrated fingertip force sensors

With integrated force sensors in the fingertip, we could use a cascade control approach. Since the motor torque and force are proportional to the armature current, we could then define the reference current r_{I_a} as a PI controller with the reference force r_F and the measured force F_{meas} as input:

$$r_{I_a}(t) = K_p(r_F - F_{meas}(t)) + K_i \int_{t_0}^t (r_F - F_{meas}(\tau)) d\tau. \quad (4.17)$$

TODO: Add tests for torque control with constant friction. Show with only measuring current, and with the force sensors.

4.3 Model reference adaptive control (MRAC) for position control

When the hand is not in contact with any objects, we want to control the position of the hand. However, due to the big differences in mass, stiffness and dampening in the hand between patients, it becomes difficult to design a controller that can guarantee good performance for all patients. To address this issue, we will implement a model reference adaptive control (MRAC) scheme, which will allow us to adapt the controller parameters online based on the observed behavior of the system. To be clear, this is mostly just to speed up the tuning process of the controller, and get good enough control for tests when frequently switching between patients. I would however recommend to

use a simpler control approach if the system is to be used several times by the same patient. This will be discussed in more detail in [TODO: Add reference to chapter: discussion].

As discussed in Section 3.2.3, because of modelling uncertainties and the fact that we don't measure anything at the hand, the finger model from Section 3.1 becomes unobservable. Therefore controlling the angles of the fingers directly becomes infeasible. Instead we will focus on controlling the position of the motor precisely. For this we will use the MRAC scheme introduced in Section 2.3.3. It calls for the system to be controllable. Using the simplified model of the finger dynamics as a torsional mass-spring-damper system acting on the motor, from Section 3.2.4, we can check if the system is controllable.

4.3.1 Controllability of the system

Based on the state-space representation our system in Section 3.2.4, we can check if the system is controllable, as described in Section 2.3.2. The controllability matrix $\mathcal{C}$ for our system is given by:

$$\mathcal{C} = [\mathbf{B}, \mathbf{AB}]. \quad (4.18)$$

Substituting for our specific $\mathbf{A}$ and $\mathbf{B}$, we get:

$$\mathcal{C} = \begin{bmatrix} 0 & \frac{K_t}{J_m + J} \\ \frac{K_t}{J_m + J} & -\frac{K_t(B_m + c)}{(J_m + J)^2} \end{bmatrix}. \quad (4.19)$$

Clearly, $\text{rank}(\mathcal{C}) = 2$, which is equal to the number of states. Therefore, the simplified system is controllable.

4.3.2 MRAC implementation

The MRAC scheme that will be implemented is based on Section 2.3.3. In our physical model of the system, we will use the finger model developed in Section 3.1, connected to the brushed DC motor as a load, through a cable connected to a whiffle tree to spread the force to all three phalanges. From the motor side, the dynamics can be described through the state-space representation:

$$\underbrace{\frac{d}{dt} \begin{bmatrix} \theta \\ \omega \end{bmatrix}}_{\dot{\mathbf{x}}} = \underbrace{\begin{bmatrix} 0 & 1 \\ 0 & -\frac{B_m}{J_m} \end{bmatrix}}_{\mathbf{A}} \underbrace{\begin{bmatrix} \theta \\ \omega \end{bmatrix}}_{\mathbf{x}} + \underbrace{\begin{bmatrix} 0 \\ \frac{K_t}{J_m} \end{bmatrix}}_{\mathbf{B}} \underbrace{I_a}_u + \underbrace{\begin{bmatrix} 0 \\ \tau_l \end{bmatrix}}_{\mathbf{d}}, \quad y(t) = \underbrace{\begin{bmatrix} 1 & 0 \end{bmatrix}}_{\mathbf{C}} \underbrace{\begin{bmatrix} \theta \\ \omega \end{bmatrix}}_{\mathbf{x}}, \quad (4.20)$$

where θ is the angle of the motor, ω is the angular velocity of the motor, I_a is the

armature current, and τ_l is the load torque applied to the motor by the finger. The output $y(t)$ is the angle of the motor, which we want to control. The MRAC scheme calls for a reference model, which the system should follow as closely as possible. Since our system is a second-order system, we will use a second-order reference model, in the standard form:

$$\underbrace{\frac{d}{dt} \begin{bmatrix} \theta_m \\ \omega_m \end{bmatrix}}_{\mathbf{\dot{x}}_m} = \underbrace{\begin{bmatrix} 0 & 1 \\ -\omega_n^2 & -2\zeta\omega_n \end{bmatrix}}_{\mathbf{A}_m} \underbrace{\begin{bmatrix} \theta_m \\ \omega_m \end{bmatrix}}_{\mathbf{x}_m} + \underbrace{\begin{bmatrix} 0 \\ \omega_n^2 \end{bmatrix}}_{\mathbf{B}_m} r_\theta, \quad (4.21)$$

where ω_n is the natural frequency of the system, ζ is the damping ratio, and r_θ is the reference angle for the motor that we want the system to follow. The MRAC scheme requires that the reference model is stable. The eigenvalues of $\mathbf{A}_m$ in Equation (4.21) are given by:

$$\lambda = -\zeta\omega_n \pm \omega_n\sqrt{\zeta^2 - 1}. \quad (4.22)$$

Therefore, as long as $\zeta, \omega_n > 0$, the reference model will be stable. Similarly to the torque controller, we'll use the current controller designed in Section 4.1 to control the armature current I_a , so that we can simply treat I_a as the control input for the MRAC scheme. We can therefore define the reference current r_{I_a} as:

$$r_{I_a}(t) = -\mathbf{K}^T(t)\mathbf{x}(t) + L(t)r_\theta(t), \quad (4.23)$$

where $\mathbf{K}(t) \in \mathbb{R}^2$ and $L(t) \in \mathbb{R}$ are the adaptive parameters that will be updated online based on the observed behavior of the system. Defining the tracking error $\mathbf{e}(t)$ as:

$$\mathbf{e}(t) = \mathbf{x}(t) - \mathbf{x}_m(t). \quad (4.24)$$

Solving for $\mathbf{P}$ for a $\mathbf{Q} = \mathbf{Q}^T > 0$ in the Lyapunov equation:

$$\mathbf{A}_m^T \mathbf{P} + \mathbf{P} \mathbf{A}_m = -\mathbf{Q}, \quad (4.25)$$

we can define the adaptive law as:

$$\begin{aligned} \dot{\mathbf{K}}(t) &= \mathbf{\Gamma}_K \mathbf{x} \mathbf{B}_m^T \mathbf{P} \mathbf{e}, \\ \dot{L}(t) &= -\gamma_L \mathbf{B}_m^T \mathbf{P} \mathbf{e} r_\theta(t), \end{aligned} \quad (4.26)$$

where $\mathbf{\Gamma}_K \in \mathbb{R}^{2 \times 2}$ and $\gamma_L \in \mathbb{R}$ are positive definite matrices that determine the adaptation rate of the parameters.

4.3.3 Simulation of MRAC controller with finger dynamics

For the simulation of the position controller, we will simulate two motors with the dynamics from Equation (4.20). One for flexion and one for extension, as is the case in our physical system. The force the motors applies on each of their cables gets distributed to the three phalanges through their own whiffle tree, with $w_1 = \frac{1}{2}$ of the force going to the proximal phalanx, $w_2 = \frac{1}{3}$ of the force going to the middle phalanx, and $w_3 = \frac{1}{6}$ of the force going to the distal phalanx. The force application is shown in Figure 4.6.

4.3.4 Cable tension

To emulate the cable connecting the motor to the finger, the following constraint is added to the system:

$$g(\theta, \mathbf{q}) \triangleq r_{spindle}\theta + L_{finger}(\mathbf{q}) - C = 0, \quad \mathbf{q} = \begin{bmatrix} \varphi_{MCP} \\ \varphi_{PIP} \\ \varphi_{DIP} \end{bmatrix}, \quad (4.27)$$

where $r_{spindle}$ is the radius of the spindle mounted on the motor, θ is the angle of the motor, $L_{finger}(\mathbf{q})$ is an estimate for the total length of cable from the finger to the whiffle tree. C is a constant defining the total cable length in the system, which then constraints the angle of the motor and the relative angles of the finger. The estimate of the total cable going from the finger to the whiffle tree is defined as:

$$L_{finger}(\mathbf{q}) = \sum_{i=1}^3 w_i \|\mathbf{r}_{a,i} - \mathbf{r}_{t,i}\|, \quad (4.28)$$

where $\mathbf{r}_{a,i}$ and $\mathbf{r}_{t,i}$ are the attachment and target points of the i -th phalanx, respectively, shown in Figure 4.6. The distance between the attachment and target points of each phalanx is scaled with the force ratios w_i to get a better estimate of the total length of cable with the whiffle tree. Differentiating the constraint, we get the velocity constraint:

$$\dot{g}(\omega, \mathbf{q}, \dot{\mathbf{q}}) = r_{spindle}\omega + \frac{\partial L_{finger}}{\partial \mathbf{q}} \dot{\mathbf{q}} = 0, \quad (4.29)$$

and further once more we get the acceleration constraint:

$$\ddot{g}(\dot{\omega}, \mathbf{q}, \dot{\mathbf{q}}, \ddot{\mathbf{q}}) = r_{spindle}\dot{\omega} + \frac{\partial L_{finger}}{\partial \mathbf{q}} \ddot{\mathbf{q}} + \dot{\mathbf{q}}^T \frac{\partial^2 L_{finger}}{\partial \mathbf{q}^2} \dot{\mathbf{q}} = 0. \quad (4.30)$$

To reduce numerical drift, Baumgarte stabilization is added, to combine all the constraints into a single equation:

$$\ddot{g} + 2\alpha\dot{g} + \beta^2 g = 0, \quad \alpha, \beta > 0. \quad (4.31)$$

The final constraint in Equation (4.31) is defined separately for the flexion and extension cables. The corresponding Lagrange multipliers are treated as the cable tensions T_f and T_e for the flexion and extension cable, respectively, and are solved simultaneously at each timestep.

Since the cables only can pull on the finger, we further impose that the cable tensions must be non-negative:

$$T_f, T_e \geq 0. \quad (4.32)$$

To be clear, this implementation of the cable tension is not a perfect emulation of the real system. In reality, the cable is slightly elastic, and the whiffle trees will provide some friction. There will also be more cable in the system. However, considering that the position of the attachment points, target points, and whiffle trees are not yet decided in the real system, this should be a good enough approximation to test the position controller, and get a better understanding of the dynamics of the system.

4.3.5 Updated dynamics with the cable tension

Given the cable tension T_x for the x -th cable, the x -th brushed DC motor experiences a load torque of $\tau_l = r_{spindle}T_x$, giving updated mechanical dynamics from Equation (3.18):

$$J_m \frac{d^2\theta_x}{dt^2} + B_m \frac{d\theta_x}{dt} = K_t I_a - r_{spindle}T_x. \quad (4.33)$$

Similarly, the finger experiences a generalized load torque. Since the cable tension acts along the line from the attachment point $\mathbf{r}_{a,i}$ to the target point $\mathbf{r}_{t,i}$, the force applied to the i -th phalanx is

$$\mathbf{F}_{x,i} = w_i T_x \frac{\mathbf{r}_{x,t,i} - \mathbf{r}_{x,a,i}}{\|\mathbf{r}_{x,t,i} - \mathbf{r}_{x,a,i}\|}. \quad (4.34)$$

The generalized torque contribution from this force is computed with:

$$\boldsymbol{\tau}_{x,i} = \frac{\partial(\mathbf{r}_{x,a,i} - \mathbf{r}_{x,t,i})^T}{\partial \mathbf{q}} \mathbf{F}_{x,i}, \quad (4.35)$$

which leads to the total generalized cable torque on the finger:

$$\boldsymbol{\tau}_{cable} = \sum_{i=1}^3 \boldsymbol{\tau}_{x,i}. \quad (4.36)$$

Given that we have two motors pulling on the finger with their own cables, we can calculate the total generalized torque from both the flexion and extension cables on the finger as:

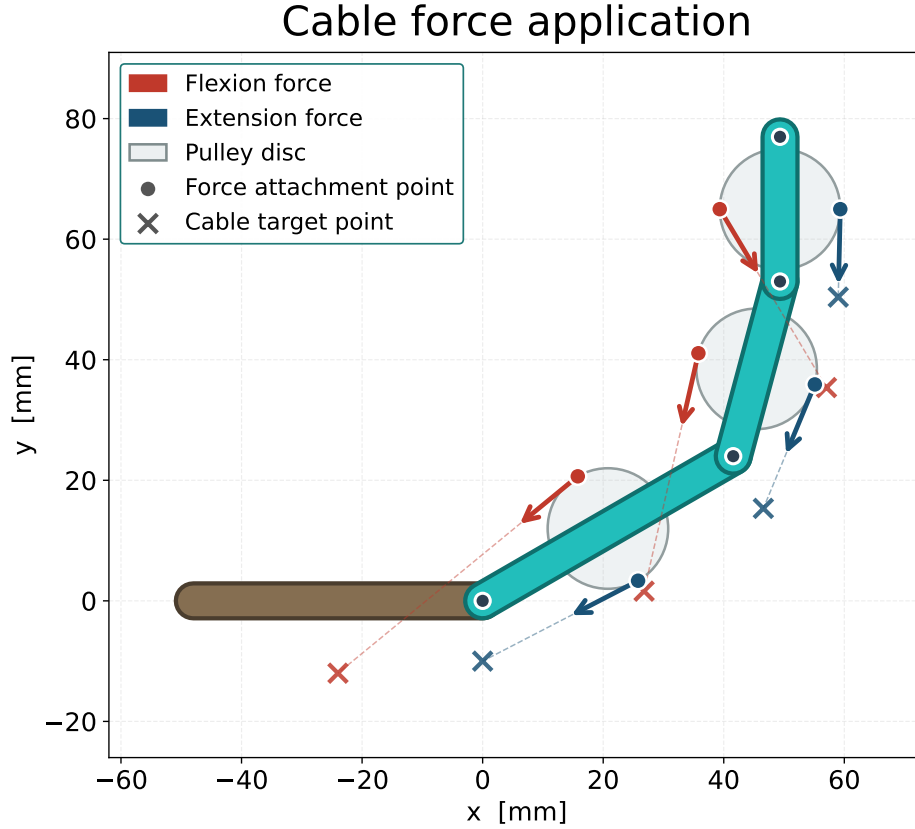


Figure 4.6: Force application on the finger. The flexion motor applies a force F_f on the flexion cable, which gets distributed to the three phalanges through the whiffle tree. Similarly, the extension motor applies a force F_e on the extension cable, which also gets distributed to the three phalanges through its own whiffle tree.

$$\tau_{cable} = \sum_{i=1}^3 \frac{\partial(\mathbf{r}_{f,a,i} - \mathbf{r}_{f,t,i})^T}{\partial \mathbf{q}} \mathbf{F}_{f,i} + \sum_{i=1}^3 \frac{\partial(\mathbf{r}_{e,a,i} - \mathbf{r}_{e,t,i})^T}{\partial \mathbf{q}} \mathbf{F}_{e,i}. \quad (4.37)$$

This gives us the updated finger dynamics from Equation (3.15):

$$\mathbf{M}(\mathbf{q})\ddot{\mathbf{q}} + (\mathbf{C}(\mathbf{q}, \dot{\mathbf{q}}) + \mathbf{B})\dot{\mathbf{q}} + \mathbf{K}(\mathbf{q} - \mathbf{q}_{eq}) = \boldsymbol{\tau}_{ext} + \boldsymbol{\tau}_{cable}. \quad (4.38)$$

4.3.6 Evaluating model reference adaptive controller

The finger model from Section 3.1 is used to simulate the dynamics of the finger in response to the force, and to calculate the force it puts on the cables. The parameters for the motor and finger model are given in [TODO: Add table of model parameters]. Both cables are initialized to be under tension when the finger is in a straight position, i.e. when $\varphi_{MCP} = \varphi_{PIP} = \varphi_{DIP} = 0$. To make sure the motors aren't working against each other, they receive an opposite reference angle when the controller is active. This ensures that as one motor pulls on the finger, increasing the tension in its cable, the other motor will be pulling in the opposite direction, decreasing the tension in its cable.

Bibliography

- National Cancer Institute. (2026). *Anatomical Terminology / SEER Training*. Retrieved March 13, 2026, from <https://training.seer.cancer.gov/anatomy/body/terminology.html>
- Rahman, N., & Mat Yahya, N. (2021). A mathematical model of a brushed DC motor system. *Data Analytics and Applied Mathematics (DAAM)*, 2, 60–68. <https://doi.org/10.15282/daam.v2i2.6830>
- TeachMeAnatomy. (2026a). *Anatomical terms of location*. Retrieved March 14, 2026, from <https://teachmeanatomy.info/the-basics/anatomical-terminology/terms-of-location/>
- TeachMeAnatomy. (2026b). *Anatomical terms of movement*. Retrieved March 15, 2026, from <https://teachmeanatomy.info/the-basics/anatomical-terminology/terms-of-movement/>